

science education is fulfilling, it's not always directly relevant to software development. I joke (but only half so) that big O notation makes up about 80 percent of the usefulness of my degree. This chapter provides an introduction to this practical topic.

The timeit Module

“Premature optimization is the root of all evil” is a common saying in software development. (It's often attributed to computer scientist Donald Knuth, who attributes it to computer scientist Tony Hoare. Tony Hoare, in turn, attributes it to Donald Knuth.) *Premature optimization*, or optimizing before knowing what needs to be optimized, often manifests itself when programmers try to use clever tricks to save memory or write faster code. For example, one of these tricks is using the *XOR algorithm* to swap two integer values without using a third, temporary variable:

```
>>> a, b = 42, 101 # Set up the two variables.
>>> print(a, b)
42 101
>>> # A series of ^ XOR operations will end up swapping their values:
>>> a = a ^ b
>>> b = a ^ b
>>> a = a ^ b
>>> print(a, b) # The values are now swapped.
101 42
```

Unless you're unfamiliar with the XOR algorithm (which uses the `^` bitwise operator), this code looks cryptic. The problem with using clever programming tricks is that they can produce complicated, unreadable code. Recall that one of the Zen of Python tenets is *readability counts*.

Even worse, your clever trick might turn out not to be so clever. You can't just assume a crafty trick is faster or that the old code it's replacing was even all that slow to begin with. The only way to find out is by measuring and comparing the *runtime*: the amount of time it takes to run a program or piece of code. Keep in mind that increasing the runtime means the program is slowing down: the program is taking more time to do the same amount of work. (We also sometimes use the term *runtime* to refer to the period during which the program is running. *This error happened at*